

KEDA

Kubernetes Event-Driven Autoscaler

A practical guide — Beginner to Expert

Based on KEDA v2.19 · CNCF Graduated Project

Table of Contents

1. Introduction — What is KEDA?
2. Why KEDA? — Problems It Solves
3. Architecture & Core Components
4. Installation & Deployment
5. The ScaledObject CRD — Your Scaling Contract
6. Scalers & Triggers — The Catalog
7. Authentication — TriggerAuthentication
8. Real-World Example 1: RabbitMQ Order Processor
9. Real-World Example 2: Kafka Consumer Group
10. Real-World Example 3: Prometheus Custom Metric
11. Real-World Example 4: AWS SQS Worker
12. Real-World Example 5: Cron-based Pre-warming
13. Real-World Example 6: HTTP Request Rate via Prometheus
14. Scaling Jobs — The ScaledJob CRD
15. Advanced Features
16. Production Best Practices
17. Troubleshooting & Debugging
18. Observability & Metrics
19. KEDA vs. HPA vs. VPA vs. Cluster Autoscaler
20. Common Pitfalls (and How to Avoid Them)
21. Further Learning & References

1. Introduction — What is KEDA?

KEDA (Kubernetes Event-Driven Autoscaler) is a lightweight, single-purpose component that lets your Kubernetes workloads scale automatically based on real-world events — not just CPU and memory. It was created by Microsoft and Red Hat in 2019 and is a CNCF Graduated project (the same maturity level as Kubernetes itself, Prometheus, and Envoy).

In plain language: if you have ever wished your deployment could scale up when there are 10,000 messages waiting in a RabbitMQ queue, scale down to zero pods when the queue is empty overnight, or spin up workers when Prometheus reports a latency spike — that is exactly what KEDA does.

The one-sentence definition



KEDA in one sentence

KEDA is a Kubernetes operator that watches external event sources (queues, databases, metrics systems, cloud services) and drives the built-in Horizontal Pod Autoscaler to scale your workloads — including the ability to scale all the way down to zero pods.

Key characteristics

- **Lightweight:** A single operator, a metrics adapter, and an admission webhook. No service mesh, no agents on every node.
- **Non-invasive:** KEDA does not replace the HPA — it extends it. Your existing CPU-based HPAs keep working untouched.
- **Vendor-neutral:** Works on any conformant Kubernetes cluster: EKS, GKE, AKS, OpenShift, Rancher, k3s, kind, even Minikube.
- **Pluggable:** Over 70 built-in scalers and a stable interface for writing your own external scalers in any language.
- **Scale to zero:** The headline feature. Pods cost money; idle pods cost money for no reason. KEDA fixes that.

Where it sits in the stack

KEDA does not see your traffic. It does not proxy requests. It is a control-plane component: every polling interval (30 seconds by default) it asks an external source "how busy are you?" and translates the answer into a number of replicas the HPA should target. That separation is what makes it both simple and trustworthy in production.

2. Why KEDA? — Problems It Solves

Stock Kubernetes ships with three autoscalers: the Horizontal Pod Autoscaler (HPA) which scales pods horizontally, the Vertical Pod Autoscaler (VPA) which resizes pods, and the Cluster Autoscaler which adds and removes nodes. All three are built around two metrics: CPU and memory. That works fine for compute-bound workloads, but it falls apart the moment your bottleneck is not CPU.

Concrete failure modes of CPU-only autoscaling

Problem 1 — The lazy consumer

You have a worker that reads messages from RabbitMQ. Each message triggers a five-second call to a slow third-party API. The worker spends 99% of its time waiting on I/O, so CPU stays near zero. The HPA looks at 5% CPU usage and concludes everything is fine. Meanwhile your queue has 50,000 messages waiting and your customers are complaining.

Problem 2 — The bursty event source

A retail site runs a flash sale at 09:00. At 08:59 traffic is nearly zero; at 09:00 it explodes by a factor of 1,000x. The HPA reacts to CPU after the surge has already arrived, the system thrashes, and you spend 90 seconds catching up while customers see error pages.

Problem 3 — The midnight idle bill

Your background email-sender service runs 24/7. At night it has nothing to do, but you keep two replicas warm "just in case". That is 24 pod-hours per day of waste, every day, in every environment (dev, staging, prod). Across an org with 200 such services, the cost is real money.

Problem 4 — The custom-metric blind spot

You want to scale based on database connection-pool saturation, Kafka consumer lag, the number of pending CI jobs in GitLab, or the depth of an Azure Service Bus subscription. None of those are CPU. The standard HPA cannot read them without you wiring up the Kubernetes custom metrics API yourself — which is its own multi-week project.

TIP

KEDA solves all four of these directly. It scales on the real signal (queue length, lag, request rate), it supports scale-to-zero, and it ships 70+ pre-built scalers so you almost never have to write metric-collection code.

Use cases at a glance

Scenario	Signal KEDA Watches	Typical Scaler
Async order processing	Queue depth	RabbitMQ / SQS / Service Bus
Stream processing	Consumer lag	Kafka / Kinesis / Pulsar
API gateway under load	Request rate or p95 latency	Prometheus / Datadog
Batch ETL jobs	Pending row count	PostgreSQL / MSSQL / MongoDB
CI/CD runners	Pending pipeline jobs	GitHub Runner / Azure Pipelines
Scheduled pre-warm	Time of day	Cron
Cost-driven scale-down	Any metric → zero overnight	Any + idleReplicaCount

Scenario	Signal KEDA Watches	Typical Scaler
ML inference	Request queue length	HTTP add-on / custom
Browser test grid	Pending sessions	Selenium Grid
Log volume spike	Log query count	Loki / Elasticsearch / Splunk

3. Architecture & Core Components

KEDA is intentionally small. Four moving parts, three CRDs, one job: turn external signals into HPA scaling decisions.

The four runtime components

1. The Operator (keda-operator)

A Deployment running in the keda namespace. It watches the cluster for ScaledObject and ScaledJob resources. When it sees one, it creates a matching HPA (for ScaledObject) or starts a Job-spawning loop (for ScaledJob). It also handles the activation phase — the 0-to-1 transition that the standard HPA cannot do on its own.

2. The Metrics Adapter (keda-operator-metrics-apiserver)

A second Deployment that implements the Kubernetes external metrics API. When the HPA asks Kubernetes "what is the current value of rabbitmq-queue-orders?", the request is routed here. The adapter queries the appropriate scaler, returns the number, and the HPA does its normal math.

3. The Admission Webhook (keda-admission-webhooks)

A validating webhook that catches misconfigurations before they reach the cluster — for example, two ScaledObjects pointing at the same Deployment, which would cause both HPAs to fight for control. Introduced in v2.10; strongly recommended in production.

4. Scalers

Not separate processes — they are Go packages compiled into the operator and metrics adapter. Each scaler knows how to talk to one event source (Kafka, SQS, Prometheus, etc.) and exposes two functions: IsActive (used for 0 → 1 activation) and GetMetrics (used for 1 → N scaling).

The three CRDs you will use

CRD	What It Does	When To Use
ScaledObject	Links a Deployment / StatefulSet / Custom Resource to one or more triggers	Long-running services (most common — 95% of cases)
ScaledJob	Spawns a Kubernetes Job per event	Long-running, one-shot batch tasks where a pod restart mid-work is unacceptable
TriggerAuthentication	Bundles credentials (secrets, IAM roles, Vault paths) for a trigger to use	Any trigger that needs to authenticate (almost all of them)

The scaling loop, step by step

1. You apply a ScaledObject pointing at Deployment "worker" with one trigger: RabbitMQ queue "orders" at threshold 5.
2. KEDA creates an HPA named keda-hpa-<your-scaledobject-name> targeting that Deployment.
3. Every 30 seconds (pollingInterval), the operator queries RabbitMQ for queue length.
4. If the queue has > 0 messages and Deployment is at 0 replicas, the operator scales it directly to minReplicaCount (1 by default).

5. Once Deployment is ≥ 1 replica, the HPA takes over: it reads the queue length via the metrics adapter and computes target replicas = $\text{ceil}(\text{queueLength} / 5)$.
6. When the queue is empty for cooldownPeriod (300 seconds), the operator scales Deployment back to 0.

**NOTE**

The activation ($0 \leftrightarrow 1$) is handled by KEDA. The scaling ($1 \leftrightarrow N$) is handled by the standard HPA. KEDA does NOT replace the HPA — it feeds it.

4. Installation & Deployment

KEDA installs in under a minute. You have three reasonable choices: Helm (recommended for most clusters), the official YAML manifests (good for GitOps workflows), or the operator marketplaces (OperatorHub on OpenShift, Azure Marketplace for AKS managed-add-on).

Option A — Helm (recommended)

```
# Add the chart repo
helm repo add kedacore https://kedacore.github.io/charts
helm repo update

# Install into a dedicated namespace
kubectl create namespace keda
helm install keda kedacore/keda --namespace keda

# Verify
kubectl get pods -n keda
# Expected: 3 pods Running
# keda-operator-xxxxxxxxxx-xxxxx
# keda-operator-metrics-apiserver-xxxxxxxxxx-xxxxx
# keda-admission-webhooks-xxxxxxxxxx-xxxxx
```

Option B — Plain YAML

```
# Latest stable release
kubectl apply --server-side -f \
  https://github.com/kedacore/keda/releases/download/v2.19.0/keda-2.19.0.yaml
```

NOTE

Use --server-side because the CRDs are large and exceed the default client-side apply size limit.

Option C — Cloud-managed

- AKS: `az aks update --enable-keda` (Microsoft maintains the install)
- OpenShift: install "Custom Metrics Autoscaler Operator" from OperatorHub (Red Hat re-brands KEDA)
- GKE: no native managed option — use Helm
- EKS: no native managed option — use Helm

Uninstall

```
# If installed via Helm:
helm uninstall keda -n keda
kubectl delete namespace keda

# If installed via YAML:
kubectl delete -f https://github.com/kedacore/keda/releases/download/v2.19.0/keda-
```



```
2.19.0.yaml
```

⚠ WARNING

Uninstalling KEDA deletes the metrics-server deployment, which means any HPA created by KEDA stops getting metrics.

Before you uninstall, decide what should happen to your scaled workloads — they will freeze at their current replica count.

5. The ScaledObject CRD — Your Scaling Contract

The ScaledObject is where you tell KEDA: "scale this thing, based on this signal, between these limits." Everything else is configuration. This section walks every field with explanation.

Minimum viable ScaledObject

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: my-worker-scaler
  namespace: production
spec:
  scaleTargetRef:
    name: my-worker           # Name of the Deployment to scale
  minReplicaCount: 0         # Scale to zero when idle
  maxReplicaCount: 30        # Hard ceiling
  triggers:
  - type: rabbitmq
    metadata:
      protocol: amqp
      queueName: orders
      mode: QueueLength
      value: "5"
      host: amqp://user:pass@rabbit.svc:5672/
```

Apply that, and KEDA does the rest. Now let us understand every knob you might turn.

Field-by-field reference

Field	Default	What It Controls
scaleTargetRef.name	(required)	The Deployment/StatefulSet/CR to scale
scaleTargetRef.kind	Deployment	Target kind (Deployment, StatefulSet, ...)
scaleTargetRef.apiVersion	apps/v1	API version of the target
pollingInterval	30 seconds	How often KEDA queries the source while idle
cooldownPeriod	300 seconds	How long to wait empty before scaling to zero
initialCooldownPeriod	0 seconds	Grace period after ScaledObject creation
minReplicaCount	0	Minimum replicas (set 1+ to disable scale-to-zero)
maxReplicaCount	100	Hard upper bound — KEDA never exceeds this

Field	Default	What It Controls
idleReplicaCount	(ignored)	Sit at N idle, jump to minReplicaCount on activity
fallback	(none)	Replica count to use when the scaler errors
advanced.restoreToOriginalReplicaCount	false	On ScaledObject deletion, restore pre-KEDA count
advanced.horizontalPodAutoscalerConfig.behavior	(HPA defaults)	Pass-through to HPA scaleUp/scaleDown behavior
advanced.scalingModifiers	(none)	Combine multiple triggers via a formula
triggers	(required)	List of one or more event sources

pollingInterval — how fast does KEDA react?

Default is 30 seconds. While the workload is at zero replicas, KEDA itself polls; this is the relevant value. Once at 1+ replicas, the standard HPA control loop takes over (default ~15 seconds, controlled by kube-controller-manager). A shorter interval reacts faster but puts more load on the source system. Anything under 10 seconds is almost always a mistake.

cooldownPeriod — protect against flapping

After the trigger reports "inactive", KEDA waits cooldownPeriod seconds before scaling to zero. The default 300 seconds (5 minutes) is sensible for most queue-driven workloads — short enough to save cost, long enough to avoid scaling down right before another batch arrives. For very bursty workloads where the cost of a cold start matters more than the cost of an idle pod, raise this to 600–900 seconds.

minReplicaCount and idleReplicaCount

These two fields are the source of more confusion than any other field. Read carefully:

- **minReplicaCount = 0:** Classic scale-to-zero. When no activity, pods = 0. When activity arrives, pods jump to 1 and HPA can scale them further up to maxReplicaCount.
- **minReplicaCount = 3:** Never go below 3 pods. Use when cold-start latency is unacceptable. HPA scales between 3 and maxReplicaCount.
- **idleReplicaCount = 0, minReplicaCount = 3:** Sit at 0 when idle, jump straight to 3 (not 1) when activity starts. Useful for warm pools that need a minimum quorum.

⚠ WARNING

idleReplicaCount only supports the value 0 due to an HPA-controller limitation. If you want a stable warm pool, set minReplicaCount to the warm-pool size and omit idleReplicaCount entirely.

fallback — what happens when the scaler itself breaks?

What if your Prometheus server goes down? Without fallback, KEDA propagates no metric to the HPA, the HPA gets confused, and behavior becomes implementation-defined. With fallback, you say "after N consecutive failures, pretend the metric value is X" — which deterministically pins the deployment at a safe replica count.

spec:

```
fallback:
  failureThreshold: 3      # After 3 consecutive scaler errors...
  replicas: 6              # ...scale to this count and hold there
  behavior: static         # static | currentReplicas |
                           # currentReplicasIfHigher | currentReplicasIfLower
```

**TIP**

Fallback is supported for all triggers EXCEPT CPU, memory, and ScaledJobs. Always set it for triggers that depend on remote services you do not control (cloud APIs, third-party SaaS).

advanced.horizontalPodAutoscalerConfig.behavior

KEDA passes this section through to the generated HPA. This is the same behavior block you would use on a vanilla HPA — useful for slowing scale-down to absorb traffic dips without thrashing:

```
spec:
  advanced:
    horizontalPodAutoscalerConfig:
      behavior:
        scaleDown:
          stabilizationWindowSeconds: 300  # Wait 5 min before each scale-down
          policies:
            - type: Percent
              value: 10                    # Remove at most 10% of pods...
              periodSeconds: 60            # ...per minute
        scaleUp:
          stabilizationWindowSeconds: 0    # Scale up immediately
          policies:
            - type: Percent
              value: 100                   # Double pods...
              periodSeconds: 15            # ...every 15 seconds
```

6. Scalers & Triggers — The Catalog

A scaler is the bit of code that knows how to ask one specific system "how busy are you?". KEDA v2.19 ships with 70+ scalers across queues, streams, databases, metrics systems, cloud services, and orchestrators. You add them to a ScaledObject under the triggers field. A single ScaledObject can have multiple triggers, and by default KEDA scales to satisfy whichever trigger asks for the most pods.

Built-in scalers — the categories

Message queues & event buses

Apache Kafka, Apache Pulsar, RabbitMQ, NATS JetStream, ActiveMQ, ActiveMQ Artemis, IBM MQ, Beanstalkd, NSQ, Solace PubSub+, Liiklus.

Cloud messaging / streaming

AWS SQS, AWS Kinesis, AWS DynamoDB Streams; Azure Service Bus, Azure Event Hubs, Azure Storage Queue, Azure Blob Storage; GCP Pub/Sub, GCP Cloud Tasks, GCP Storage.

Databases

PostgreSQL, MySQL, MSSQL, MongoDB, CouchDB, Cassandra, ArangoDB, Etcd, AWS DynamoDB.

Caches & in-memory stores

Redis Lists, Redis Streams (with Sentinel and Cluster variants for both).

Observability / metrics systems

Prometheus, Datadog, New Relic, Dynatrace, Splunk, Splunk Observability, Elasticsearch, Loki, Graphite, InfluxDB, Sumo Logic, SolarWinds, Azure Monitor, Azure Application Insights, Azure Log Analytics, Azure Data Explorer, AWS CloudWatch, GCP Stackdriver, Huawei Cloudeye, OpenStack Metric, OpenStack Swift.

CI / build / runners

GitHub Runner, Forgejo, Azure Pipelines, Selenium Grid.

Kubernetes-native

CPU, Memory, Kubernetes Resource (any K8s API metric), Kubernetes Workload (pod count of another workload), Cron (time-of-day).

Generic / extension points

Metrics API (any HTTP endpoint returning JSON), External (gRPC pull), External Push (gRPC push), Predictkube (ML-based prediction), Temporal.

Multiple triggers on one ScaledObject

You can put any number of triggers on the same ScaledObject. By default the OR semantics apply: KEDA scales to whichever count is highest across all triggers. You can override this with `advanced.scalingModifiers` — see section 15.

```
spec:
  triggers:
    - type: rabbitmq
      metadata: { queueName: orders, value: "5", host: amqp://... }
      # Trigger 1: queue depth
    - type: prometheus
      # Trigger 2: p95 latency
```

```
metadata:  
  serverAddress: http://prometheus:9090  
  metricName: http_latency_p95  
  threshold: "500"  
  query: histogram_quantile(0.95, http_request_duration_seconds_bucket)
```

In that example: if the queue is empty but latency is high, KEDA still scales up. If the queue is deep but latency is fine, KEDA still scales up. The pod count is the maximum the two triggers ask for.

7. Authentication — TriggerAuthentication

Hardcoding credentials in a ScaledObject would be terrible practice and most production scalers refuse to do it. Instead, KEDA defines TriggerAuthentication (namespaced) and ClusterTriggerAuthentication (cluster-wide) resources that bundle credentials from one of several providers, and you reference them by name from your trigger.

Supported credential providers

Provider	Use Case
Secret	Pull username/password/token from a Kubernetes Secret (most common)
ConfigMap	Pull non-sensitive fields (URLs, region) from a ConfigMap
Environment variable	Read from the target Deployment's env (handy when the app already has creds)
File path	Read from a mounted file (e.g., a CSI-driver mounted secret)
AWS (IRSA)	IAM Roles for Service Accounts on EKS — no static keys
AWS EKS Pod Identity	Newer EKS-native identity flow
AWS Secret Manager	Pull credentials from AWS Secret Manager at scale-time
Azure AD Workload Identity	OIDC-federated identities on AKS — the modern Azure way
Azure Key Vault	Pull credentials from a Key Vault
GCP Workload Identity	Bind a Kubernetes ServiceAccount to a GCP IAM service account
GCP Secret Manager	Pull from GCP Secret Manager
Hashicorp Vault	Pull from any Vault path (KV v1/v2, AWS engine, etc.)
Bound service account token	OIDC-style token mounted into the operator pod

Example: TriggerAuthentication backed by a Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: rabbit-creds
  namespace: production
type: Opaque
stringData:
  host: amqp://user:s3cret@rabbit.production.svc:5672/
```

```
---
apiVersion: keda.sh/v1alpha1
kind: TriggerAuthentication
metadata:
  name: rabbit-auth
  namespace: production
spec:
  secretTargetRef:
    - parameter: host          # Name the scaler expects
      name: rabbit-creds       # The Secret
      key: host                # The key within the Secret
---
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: order-processor
  namespace: production
spec:
  scaleTargetRef:
    name: order-processor
  triggers:
    - type: rabbitmq
      metadata:
        queueName: orders
        mode: QueueLength
        value: "10"
      authenticationRef:
        name: rabbit-auth      # ← reference here
```

**TIP**

Use ClusterTriggerAuthentication when the same credentials are reused across many namespaces — for example, a shared Vault token, or an IRSA role that scales every team's SQS workers.

Use TriggerAuthentication when credentials are namespace-scoped — which is the safer default for multi-tenant clusters.

8. Real-World Example 1: RabbitMQ Order Processor

Scenario: An e-commerce platform places every checkout into a RabbitMQ queue called orders. A worker Deployment processes them, calling a payment gateway, updating inventory, and emitting a confirmation email. Each message takes 2–4 seconds. Daily volume is 1 million messages but it is heavily bursty — 70% of traffic falls within four one-hour peaks.

Goal

- Each worker should hold about 10 in-flight messages at a time (prefetch = 10).
- Scale to zero at night when the queue is empty.
- Cap at 50 pods so we do not overwhelm the payment gateway.
- Tolerate brief RabbitMQ management-API outages (use fallback).

Full manifest

```
# 1. Secret holding RabbitMQ credentials
apiVersion: v1
kind: Secret
metadata:
  name: rabbit-creds
  namespace: production
type: Opaque
stringData:
  host: amqp://orderworker:hunter2@rabbit.production.svc.cluster.local:5672/
---
# 2. TriggerAuthentication wires the secret into KEDA
apiVersion: keda.sh/v1alpha1
kind: TriggerAuthentication
metadata:
  name: rabbit-auth
  namespace: production
spec:
  secretTargetRef:
    - parameter: host
      name: rabbit-creds
      key: host
---
# 3. The workload itself
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-processor
  namespace: production
spec:
  replicas: 0 # KEDA will set this
  selector: { matchLabels: { app: order-processor } }
  template:
    metadata: { labels: { app: order-processor } }
```

```
spec:
  terminationGracePeriodSeconds: 60    # Let in-flight messages finish
  containers:
    - name: worker
      image: registry.example.com/order-processor:1.4.2
      env:
        - { name: AMQP_URL, valueFrom: { secretKeyRef: { name: rabbit-creds, key:
host } } }
        - { name: PREFETCH_COUNT, value: "10" }
      resources:
        requests: { cpu: "100m", memory: "128Mi" }
        limits:   { cpu: "500m", memory: "256Mi" }
  ---
# 4. The ScaledObject – the interesting part
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: order-processor
  namespace: production
spec:
  scaleTargetRef:
    name: order-processor
  pollingInterval: 15                # Check queue every 15s while idle
  cooldownPeriod: 120                # Wait 2 min empty before scale-to-zero
  minReplicaCount: 0
  maxReplicaCount: 50
  fallback:
    failureThreshold: 5
    replicas: 5                    # If RabbitMQ mgmt API breaks, hold 5 pods
    behavior: currentReplicasIfHigher # ...or current count, whichever is higher
  advanced:
    horizontalPodAutoscalerConfig:
      behavior:
        scaleDown:
          stabilizationWindowSeconds: 180
          policies:
            - { type: Percent, value: 20, periodSeconds: 60 }
        scaleUp:
          stabilizationWindowSeconds: 0
          policies:
            - { type: Percent, value: 100, periodSeconds: 15 }
            - { type: Pods, value: 10, periodSeconds: 15 }
  triggers:
    - type: rabbitmq
      metricType: AverageValue
      metadata:
        protocol: amqp
        queueName: orders
```

```
mode: QueueLength
value: "10"                # Target 10 messages per pod
activationValue: "5"       # Stay at 0 until queue has > 5
authenticationRef:
  name: rabbit-auth
```

What is happening here, line by line

- **pollingInterval: 15** — When idle (replicas=0), check the queue every 15 seconds. Reactivity matters most here.
- **cooldownPeriod: 120** — 2 minutes of empty queue before we scale back to zero. Prevents flap during natural lulls.
- **mode: QueueLength, value: "10"** — HPA will compute $\text{targetReplicas} = \text{ceil}(\text{queueLength} / 10)$. So 100 messages = 10 pods.
- **activationValue: "5"** — We do not bother spinning a pod for fewer than 5 messages. Avoids cold-start churn for stragglers.
- **fallback.behavior: currentReplicasIfHigher** — If the RabbitMQ Management API breaks (it does), do not collapse — hold the current count or 5, whichever is higher.
- **scaleDown.stabilizationWindowSeconds: 180** — Wait 3 minutes between scale-downs. Stops the "I just scaled, no wait, scale back" oscillation.
- **terminationGracePeriodSeconds: 60** — Crucial: when HPA decides to remove a pod, give it 60s to finish in-flight messages before SIGKILL.

9. Real-World Example 2: Kafka Consumer Group

Scenario: A clickstream service produces events to a Kafka topic web-events with 24 partitions. A consumer group click-aggregator reads, aggregates per session, and writes to a data lake. The number of consumers in the group must be $\leq$ the number of partitions; otherwise the extras sit idle.

Goal

- Scale on consumer lag — the difference between latest offset and committed offset.
- Never exceed 24 pods (the partition count is the hard ceiling).
- Scale to zero overnight when producers are idle.

```
apiVersion: v1
kind: Secret
metadata:
  name: kafka-creds
  namespace: data
type: Opaque
stringData:
  sasl: "plaintext"
  username: "click-aggregator"
  password: "redacted"
---
apiVersion: keda.sh/v1alpha1
kind: TriggerAuthentication
metadata:
  name: kafka-auth
  namespace: data
spec:
  secretTargetRef:
    - { parameter: sasl,      name: kafka-creds, key: sasl }
    - { parameter: username, name: kafka-creds, key: username }
    - { parameter: password, name: kafka-creds, key: password }
---
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: click-aggregator
  namespace: data
spec:
  scaleTargetRef:
    name: click-aggregator
  pollingInterval: 20
  cooldownPeriod: 300
  minReplicaCount: 0
  maxReplicaCount: 24      # Match partition count exactly
  triggers:
    - type: kafka
      metadata:
```

```
bootstrapServers: kafka-broker.data.svc:9092
consumerGroup: click-aggregator
topic: web-events
lagThreshold: "1000"           # Target: 1000 unprocessed messages per pod
activationLagThreshold: "500"  # Stay at 0 until lag > 500
offsetResetPolicy: latest
allowIdleConsumers: "false"    # Cap at partition count
authenticationRef:
  name: kafka-auth
```

Why this works

Kafka assigns partitions to consumers — adding a 25th consumer to a 24-partition topic does nothing, it just sits idle. KEDA understands this via `allowIdleConsumers: "false"` and caps the effective scale at the partition count even if lag is huge. The `activationLagThreshold` prevents pod thrash when traffic trickles in — a single delayed message will not wake the whole pipeline.

WARNING

`lagThreshold` is per pod, not total. If you set `lagThreshold: "1000"` and total lag is 12,000, KEDA will request 12 pods.

Match `maxReplicaCount` to your partition count, not your CPU budget. Going higher wastes pods.

10. Real-World Example 3: Prometheus Custom Metric

Scenario: A REST API service must keep p95 latency under 500ms. CPU usage does not reliably predict latency (the bottleneck is a downstream database). Use Prometheus as the source of truth.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: catalog-api
  namespace: shop
spec:
  scaleTargetRef:
    name: catalog-api
  pollingInterval: 30
  cooldownPeriod: 300
  minReplicaCount: 2 # Never go below 2 — latency-sensitive
  maxReplicaCount: 40
  fallback:
    failureThreshold: 3
    replicas: 10
    behavior: currentReplicasIfHigher
  triggers:
  - type: prometheus
    metricType: Value # We want the raw latency, not avg per pod
    metadata:
      serverAddress: http://prometheus-server.monitoring.svc:9090
      metricName: catalog_api_p95
      threshold: "500" # Scale up to keep p95 < 500ms
      activationThreshold: "100"
      query: |
        histogram_quantile(0.95,
          sum(rate(http_request_duration_seconds_bucket{job="catalog-api"}[2m]))
            by (le)
        ) * 1000
```

Key choices

- **metricType: Value (not AverageValue)** — p95 latency is a single number for the whole fleet, not per-pod.
- **rate(...)[2m]** — Two-minute window smooths spikes that would otherwise cause scale-up oscillation.
- **minReplicaCount: 2** — No scale-to-zero. A user-facing API cannot afford a cold start.
- **fallback set** — If Prometheus dies, hold 10 pods (or current, whichever is higher). Do not collapse to 2 just because monitoring is broken.



TIP

A common mistake is to scale on average latency. Averages hide tail latency: 99 requests at 100ms and 1

request at 10s averages to 199ms but your slowest user sees 10 seconds. Always scale on p95 or p99.

11. Real-World Example 4: AWS SQS Worker (IRSA)

Scenario: A worker pulls messages from an SQS queue invoice-pdf-rendering on EKS. We do NOT want to ship AWS access keys into the cluster — instead we use IAM Roles for Service Accounts (IRSA), which gives the pod a short-lived role-assumed token via OIDC.

```
# 1. ServiceAccount annotated with the IAM role ARN
apiVersion: v1
kind: ServiceAccount
metadata:
  name: invoice-renderer
  namespace: billing
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/invoice-renderer
---
# 2. TriggerAuthentication using AWS provider
apiVersion: keda.sh/v1alpha1
kind: TriggerAuthentication
metadata:
  name: aws-irsa
  namespace: billing
spec:
  podIdentity:
    provider: aws # IRSA — KEDA picks up the role
---
# 3. The Deployment uses the same ServiceAccount
apiVersion: apps/v1
kind: Deployment
metadata:
  name: invoice-renderer
  namespace: billing
spec:
  selector: { matchLabels: { app: invoice-renderer } }
  template:
    metadata: { labels: { app: invoice-renderer } }
    spec:
      serviceAccountName: invoice-renderer
      containers:
        - name: renderer
          image: ecr.io/billing/invoice-renderer:2.1
---
# 4. ScaledObject
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: invoice-renderer
  namespace: billing
spec:
  scaleTargetRef:
```



```
    name: invoice-renderer
  minReplicaCount: 0
  maxReplicaCount: 20
  triggers:
  - type: aws-sqs-queue
    metadata:
      queueURL: https://sqs.us-east-1.amazonaws.com/123456789012/invoice-pdf-
        rendering
      queueLength: "5"
      awsRegion: us-east-1
      identityOwner: operator          # Use the operator's IRSA, not the
        workload's
      authenticationRef:
        name: aws-irsa
```

 NOTE

identityOwner: operator means KEDA itself authenticates to AWS using the role attached to the keda-operator service account. You must grant that role `sqs:GetQueueAttributes` on the queue.

identityOwner: workload (default) means KEDA assumes the role attached to the workload's service account. Use this when each workload has its own role.

12. Real-World Example 5: Cron-based Pre-warming

Scenario: A B2C app sees a 50x traffic spike Mon–Fri at 09:00 sharp (Slack-style notification dispatch). Cold-starting 50 pods at 09:00:00 means the first minute of traffic gets degraded latency. Solution: pre-warm at 08:55 by jumping minReplicaCount from 2 to 30 during the busy window.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: notification-dispatch
  namespace: production
spec:
  scaleTargetRef:
    name: notification-dispatch
  minReplicaCount: 2
  maxReplicaCount: 100
  triggers:
    # Trigger 1 – workload-driven on the actual queue
    - type: rabbitmq
      metadata:
        queueName: notifications
        mode: QueueLength
        value: "20"
        host: amqp://notify.svc:5672/
    # Trigger 2 – time-of-day pre-warm
    - type: cron
      metadata:
        timezone: Europe/London
        start: 55 8 * * 1-5           # Mon–Fri 08:55
        end: 30 10 * * 1-5           # Mon–Fri 10:30
        desiredReplicas: "30"
```

How the two triggers cooperate

KEDA takes the max across all triggers. Result: outside 08:55–10:30 the deployment scales between 2 and however-many the queue requires. During 08:55–10:30 it never falls below 30, even if the queue is empty — those 30 pods are sitting warm, ready for the 09:00 surge. After 10:30 the cron trigger goes quiet and the queue takes over again.



TIP

The Cron scaler is also great for non-production cost control: a single trigger with start: "0 19 * * *", end: "0 8 * * *", desiredReplicas: "0" effectively scales everything in your dev cluster down overnight.

13. Real-World Example 6: HTTP Request Rate via Prometheus

Scenario: An ML inference service exposes a /predict endpoint. Each request consumes ~2GB GPU memory for ~80ms. We want to scale on actual request rate, not CPU (CPU is misleading on GPU pods). Each pod can sustain ~50 req/s before latency degrades.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: model-inference
  namespace: ml
spec:
  scaleTargetRef:
    name: model-inference
  pollingInterval: 15
  cooldownPeriod: 60 # Fast scale-down – GPUs are expensive
  minReplicaCount: 1
  maxReplicaCount: 16 # GPU node ceiling
  advanced:
    horizontalPodAutoscalerConfig:
      behavior:
        scaleUp:
          policies:
            - { type: Pods, value: 4, periodSeconds: 30 } # Add up to 4 pods/30s
        scaleDown:
          stabilizationWindowSeconds: 120
          policies:
            - { type: Pods, value: 1, periodSeconds: 60 } # Remove only 1/60s
  triggers:
    - type: prometheus
      metricType: AverageValue
      metadata:
        serverAddress: http://prometheus-server.monitoring.svc:9090
        metricName: model_requests_per_sec
        threshold: "50" # 50 req/s per pod
        query: sum(rate(model_predict_requests_total[1m]))
```

The asymmetric scale policy

Scale-up is aggressive: add up to 4 pods every 30 seconds. Scale-down is conservative: remove 1 pod per minute, with a 2-minute stabilization window. This asymmetry is intentional. The cost of being late to scale up is unhappy users; the cost of being slow to scale down is a few extra GPU-minutes. Errs on the side of capacity.

14. Scaling Jobs — The ScaledJob CRD

A Deployment is the wrong shape for work that takes hours. If you scale a Deployment from 10 → 7, the HPA picks any 3 pods to kill — and one of them is probably 2.9 hours into a 3-hour render job. That work is lost. ScaledJob solves this: one event = one Kubernetes Job = one pod that runs to completion and then exits naturally. Nothing kills work in flight.

When to choose ScaledJob over ScaledObject

Choose ScaledJob if...	Choose ScaledObject if...
Each unit of work takes > 5 minutes	Work is short (seconds to a couple of minutes)
Mid-work termination would corrupt state	Workers are stateless and resumable
You want one event = one container lifecycle	You want long-running consumers with prefetch
Examples: video rendering, ML training, backups	Examples: API workers, queue consumers

Full example — RabbitMQ-driven job

```
apiVersion: keda.sh/v1alpha1
kind: ScaledJob
metadata:
  name: video-renderer
  namespace: media
spec:
  jobTargetRef:
    parallelism: 1          # One worker per Job
    completions: 1
    backoffLimit: 3        # Retry the Job up to 3 times on failure
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: renderer
        image: media/renderer:5.0
        command: ["/render", "--once"] # Render one job and exit
        resources:
          requests: { cpu: "2", memory: "4Gi" }
          limits:   { cpu: "4", memory: "8Gi" }
    pollingInterval: 10
    maxReplicaCount: 50      # Up to 50 concurrent renders
    successfulJobsHistoryLimit: 5 # Keep 5 successful Jobs for debugging
    failedJobsHistoryLimit: 10  # Keep 10 failed Jobs (always investigate)
    scalingStrategy:
      strategy: accurate      # See below
  triggers:
  - type: rabbitmq
    metadata:
      queueName: render-jobs
```

```
mode: QueueLength
value: "1"                # One pod per message
host: amqp://media-rabbit:5672/
```

scalingStrategy — the math behind job spawning

Strategy	Formula	When To Use
default	$\text{maxScale} = \text{queueLength} - \text{runningJobs}$	Conservative — good baseline
accurate	$\text{maxScale} = \text{queueLength} - (\text{runningJobs} - \text{finishedJobs})$	Long-running jobs where progress matters
eager	Spawn for every message that could possibly be in flight	Latency-sensitive work, plenty of cluster capacity
custom	You provide deduction + percentage	Fine-tuning when defaults misbehave

WARNING

ScaledJobs do NOT support the fallback field. If your trigger source is unavailable, jobs simply do not get spawned. Plan for this — alert on missing scaler metrics.

15. Advanced Features

Pausing autoscaling

During a deploy, maintenance window, or incident, you may want KEDA to stop changing replica counts without deleting the ScaledObject. Use an annotation:

```
# Pause completely, hold at current replicas
kubectl annotate scaledobject my-app autoscaling.keda.sh/paused=true

# Pause AND set replicas to an exact number
kubectl annotate scaledobject my-app autoscaling.keda.sh/paused-replicas=5

# Pause only scale-IN (allow scale-out, block scale-down)
kubectl annotate scaledobject my-app autoscaling.keda.sh/paused-scale-in=true

# Pause only scale-OUT (allow scale-down, block scale-up)
kubectl annotate scaledobject my-app autoscaling.keda.sh/paused-scale-out=true

# Resume — remove the annotation OR set to false
kubectl annotate scaledobject my-app autoscaling.keda.sh/paused-
```

Forcing activation

Want to wake a scale-to-zero workload manually (e.g., to inspect logs)?

```
kubectl annotate scaledobject my-app autoscaling.keda.sh/force-activation=true
# remove the annotation when done
kubectl annotate scaledobject my-app autoscaling.keda.sh/force-activation-
```

Scaling Modifiers — combining multiple triggers

Default behavior is "max of all triggers". Sometimes you need different logic. Scaling Modifiers let you express that with a formula using the expr language. Each trigger needs a name, and the formula references them.

Example: average of two metrics

```
spec:
  advanced:
    scalingModifiers:
      formula: "(qlen + lag) / 2"
      target: "10"
      activationTarget: "5"
      metricType: AverageValue
    triggers:
      - type: rabbitmq
        name: qlen
        metadata: { queueName: orders, ... }
      - type: kafka
```

```
name: lag
metadata: { topic: events, ... }
```

Example: weighted formula

```
scalingModifiers:
  formula: "(latency * 2) + (qlen * 1)"
  target: "100"
```

Latency-driven scaling weighted twice as heavily as queue depth. Useful when you have a primary signal and a corroborating one.

Example: conditional logic

```
scalingModifiers:
  formula: "errorRate > 0.05 ? 0 : qlen"
  target: "10"
```

A safety brake: if the error rate exceeds 5%, return 0 from the formula — which scales the deployment to its minimum. Stops a broken service from amplifying the blast radius.

Caching metrics

By default, every HPA metric query reaches through KEDA to the scaler. For expensive sources (cloud APIs with cost-per-call) this is wasteful. Enable caching to serve HPA queries from KEDA's cache, refreshed only on `pollingInterval`.

```
triggers:
- type: aws-sqs-queue
  useCachedMetrics: true           # Cuts SQS API calls dramatically
  metadata: { ... }
```

⚠ WARNING

Not supported for cpu, memory, or cron scalers.

Transferring ownership of an existing HPA

If you already manage a workload with a vanilla HPA and want to migrate to KEDA without a window of "no autoscaler", use the `transfer-hpa-ownership` annotation:

```
metadata:
  annotations:
    scaledobject.keda.sh/transfer-hpa-ownership: "true"
spec:
  advanced:
    horizontalPodAutoscalerConfig:
      name: existing-hpa-name           # Match the existing HPA exactly
```

16. Production Best Practices

Resource management

- Always set `resources.requests` on workloads KEDA scales — the cluster autoscaler needs them to know whether to add nodes.
- Set `terminationGracePeriodSeconds` appropriately. Default 30 seconds is often too short for queue workers.
- Implement a `SIGTERM` handler in your application to stop accepting new work and finish in-flight items.
- Add a `preStop` hook if you cannot modify the app code (e.g., `sleep 30; kill -TERM 1`).

Reliability

- Always set fallback on triggers that depend on remote SaaS or shared infrastructure.
- Use `minReplicaCount ≥ 2` for user-facing services. Cold starts cost more than idle pods.
- Pair KEDA with `PodDisruptionBudgets` to ensure scale-down does not violate availability SLOs.
- Spread replicas across zones with topology constraints — scaling out fast is no help if all pods land on one node.

Cost optimization

- Use scale-to-zero (`minReplicaCount: 0`) on non-critical and dev workloads — measure cold-start latency first.
- Cron-scale dev environments down overnight — KEDA pays for itself in a single billing cycle.
- Match `maxReplicaCount` to real downstream limits (DB pool size, API rate limits, partition counts).
- Enable `useCachedMetrics` on cloud-API triggers (CloudWatch, Azure Monitor) to cut API costs.

Operational hygiene

- Run the admission webhook in production — it prevents foot-gun configurations.
- Pin a specific KEDA version in your Helm values. Auto-upgrading a control-plane component is risky.
- Monitor `keda-operator` restart count — frequent restarts mean memory pressure or an event-source storm.
- Use `ClusterTriggerAuthentication` sparingly. Namespace-scoped auth is safer multi-tenancy.

GitOps integration

`ScaledObjects`, `TriggerAuthentications`, and `Secrets` all belong in Git. With Argo CD or Flux, treat `ScaledObjects` like any other manifest. One caveat: KEDA creates an HPA resource owned by the `ScaledObject`. Make sure your GitOps tool ignores those generated HPAs (in Argo CD: add the HPA to `ignoreDifferences` with `managedFieldsManagers` including `keda-operator`).

17. Troubleshooting & Debugging

The five commands you will use

```
# 1. Is the ScaledObject healthy?
kubectl describe scaledobject my-app -n production
# Look for Status.Conditions – Ready=True, Active=True/False, Fallback=False

# 2. What does KEDA think the metric value is?
kubectl get scaledobject my-app -n production -o yaml | grep -A5 status

# 3. Was the HPA created?
kubectl get hpa -n production
# Expect: keda-hpa-my-app

# 4. What is the operator complaining about?
kubectl logs -n keda -l app=keda-operator --tail=200 | grep -i error

# 5. Are external metrics flowing?
kubectl get --raw "/apis/external.metrics.k8s.io/v1beta1" | jq
```

Common problem 1 — "the deployment is not scaling"

- Check the ScaledObject's Active status. If false, the activation threshold has not been crossed.
- Check the operator logs for "scaler error" messages — the most common cause is credentials.
- Verify the HPA actually got created: `kubectl get hpa`. If absent, the ScaledObject is rejected — check describe output.
- Verify the metric is reaching the metrics adapter: `kubectl get --raw "/apis/external.metrics.k8s.io/v1beta1/namespaces/production/s0-my-trigger"`.

Common problem 2 — "the deployment scales up and down constantly"

- Increase `stabilizationWindowSeconds` on `scaleDown` (try 300).
- Raise the activation threshold so trickling traffic does not toggle 0 ↔ 1 repeatedly.
- Smooth your Prometheus query with a longer `rate()` window (2m, 5m).

Common problem 3 — "no metrics available"

- Wait 30 seconds — KEDA polls on `pollingInterval`, metrics appear after the first successful poll.
- Check that exactly one external metrics provider is installed. Conflicts with Prometheus Adapter or Datadog Cluster Agent will break KEDA.
- Restart the metrics-server pod: `kubectl rollout restart deployment keda-operator-metrics-apiserver -n keda`.

Common problem 4 — "Status.Conditions shows Fallback=True"

The scaler has been failing for `failureThreshold` consecutive polls and is using the fallback replica count. Check operator logs for the underlying error. Common causes: source unreachable, expired credentials, rate limiting, malformed query.

Useful kubectl extension

Install the keda kubectl plugin via krew for shorter commands:

```
kubectl krew install keda
kubectl keda dump           # Full diagnostic bundle for support
kubectl keda graph my-app   # ASCII-art scaling activity over time
```

18. Observability & Metrics

Prometheus integration

KEDA exposes its own Prometheus metrics endpoint (/metrics on port 8080 of the operator and metrics-server). Scrape it with a ServiceMonitor (if you run kube-prometheus-stack):

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: keda
  namespace: keda
spec:
  selector:
    matchLabels: { app: keda-operator-metrics-apiserver }
  endpoints:
    - port: metrics
      interval: 30s
```

Metrics worth alerting on

Metric	Meaning	Alert When
keda_scaler_errors_total	Scaler call failures	Increasing over 5m
keda_scaler_active	0 or 1 — is the trigger above activation?	Unexpectedly 0 during business hours
keda_scaled_object_paused	Manually paused ScaledObjects	In production for > 1h
keda_scaler_metrics_value	Current value the scaler reports	Diverges from your own metric source
keda_scaled_object_errors	Reconcile errors	> 0

CloudEvents

KEDA can emit CloudEvents on scaling actions — handy for audit trails or driving downstream automation. Enable in the operator deployment with:

```
apiVersion: eventing.keda.sh/v1alpha1
kind: CloudEventSource
metadata:
  name: keda-events
  namespace: keda
spec:
  destination:
    http:
      uri: http://audit-collector.observability.svc/keda-events
```

OpenTelemetry

KEDA 2.16+ supports OpenTelemetry metrics export. Configure the operator with `--otel-prometheus-exporter-endpoint` to push to your OTel collector. Useful in environments standardizing on OTLP over Prometheus scraping.

19. KEDA vs. HPA vs. VPA vs. Cluster Autoscaler

These tools solve different problems and they compose. Understanding the boundaries prevents arguments in code review.

Tool	Dimension	Signal	Replaces
HPA	Pods (horizontal)	CPU, memory, custom metrics	Nothing — Kubernetes built-in
VPA	Pod size (vertical)	Observed CPU/memory usage	Manual resource tuning
Cluster Autoscaler	Nodes	Pending pods that cannot be scheduled	Manual node scaling
KEDA	Pods (horizontal)	External events, queues, custom metrics, time	Nothing — it extends the HPA

How they compose

- A message arrives in RabbitMQ.
- KEDA detects it, activates the Deployment (0 → 1).
- More messages arrive; KEDA feeds the HPA, which scales 1 → 20.
- At 20 pods, there is no room on existing nodes; pods become Pending.
- Cluster Autoscaler sees Pending pods, adds nodes.
- Pods schedule, process the queue, drain it.
- KEDA detects the empty queue; HPA scales 20 → 1.
- Cooldown expires; KEDA scales 1 → 0.
- Cluster Autoscaler removes the now-unused nodes.

In that scenario, all three tools cooperate. None of them is sufficient alone.

WARNING

Do NOT use VPA in "auto" mode on workloads scaled by KEDA. VPA recreates pods when it changes their size, and KEDA + HPA do not expect their pods to be replaced underneath them. Use VPA in "off" or "recommendation" mode only.

20. Common Pitfalls (and How to Avoid Them)

Pitfall 1 — Two scalers fighting

Creating both a vanilla HPA and a KEDA ScaledObject pointing at the same Deployment causes them to overwrite each other. The admission webhook usually catches this, but if you disable it ("hpa-ownership: true") you are on your own. Always: one autoscaler per workload.

Pitfall 2 — Forgetting activation behavior

A scaler with no activation threshold defined activates the workload at metric value > 0 . For queue scalers that means a single message wakes the whole pipeline. Always set `activationValue / activationThreshold` to a sensible non-zero number.

Pitfall 3 — Queue length as Value instead of AverageValue

If you set `metricType: Value` on a queue scaler with threshold 10 and the queue has 100 items, HPA scales to 10 pods. If you set `metricType: AverageValue` (the default) with threshold 10 and queue is 100, HPA scales to $100/10 = 10$ pods too — but the math is per-pod and scales correctly as you add or remove pods. Default to `AverageValue` for queues unless you know why.

Pitfall 4 — Premature termination of long jobs

A Deployment scaled by KEDA can have any pod terminated by the HPA at any time. If your work is long-running, this means partial work is lost. Solutions: use `ScaledJob` instead, or implement `SIGTERM`-handling in your worker to drain gracefully.

Pitfall 5 — Polling interval too short

Setting `pollingInterval` to 1 second sounds fast and reactive. In practice you hammer your event source, hit rate limits, get throttled, and KEDA starts erroring out (which triggers fallback, which masks the real problem). Stick to 10–30 seconds unless you have measured a need for faster.

Pitfall 6 — Mixing TriggerAuthentication and inline credentials

Some scalers accept credentials inline in metadata as well as via `authenticationRef`. Be consistent — pick one. Inline credentials in YAML end up in Git, in logs, and in `kubectl describe` output. Use `TriggerAuthentication` for anything sensitive.

Pitfall 7 — Not testing the scale-to-zero path

Scale-to-zero only works if your application can cold-start cleanly. Test by setting `replicas: 0`, sending the activating event, and measuring how long until the first response. If it is over 30 seconds you may want `minReplicaCount: 1`.

21. Further Learning & References

This guide is a starting point — KEDA has a deep documentation tree and an active community. Here are the most useful next steps, ordered roughly from fundamentals to advanced.

Official documentation

- KEDA docs (v2.19): keda.sh/docs/2.19
- Concepts overview: keda.sh/docs/2.19/concepts
- ScaledObject spec: [reference/scaledobject-spec](https://keda.sh/docs/2.19/reference/scaledobject-spec)
- ScaledJob spec: [reference/scaledjob-spec](https://keda.sh/docs/2.19/reference/scaledjob-spec)
- Full scaler catalog: keda.sh/docs/2.19/scalers
- Authentication providers: keda.sh/docs/2.19/authentication-providers
- Official FAQ: keda.sh/docs/2.19/reference/faq
- Troubleshooting guide: keda.sh/docs/2.19/troubleshooting

Source code & community

- KEDA GitHub: github.com/kedacore/keda
- Sample manifests: github.com/kedacore/samples
- HTTP add-on (HTTP-based scaling): github.com/kedacore/http-add-on
- Kubernetes Slack #keda channel: kubernetes.slack.com/messages/CKZJ36A5D
- CNCF KEDA project page: cncf.io/projects/keda

Related Kubernetes documentation

- HPA walkthrough: kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough
- HPA configurable scaling behavior: [kubernetes.io HPA configurable behavior](https://kubernetes.io/docs/tasks/configure-pod-container/autoscale-with-hpa)
- Container lifecycle hooks: kubernetes.io/docs/concepts/containers/container-lifecycle-hooks
- PodDisruptionBudgets: kubernetes.io/docs/concepts/workloads/pods/disruptions

Talks worth watching

KEDA has a strong conference presence. Search YouTube for "KEDA KubeCon" — the maintainer talks are the single best ramp-up for production engineering. Highlights:

- "KEDA Project Update" — given at every KubeCon by Tom Kerkhove and Zbynek Roubalik. Always the current state of the project.
- "From CPU to Event-Driven: A KEDA Deep Dive" — covers architecture and the v2 generation of features.
- "Scaling to Zero — and Back" — patterns for cold-start mitigation in event-driven systems.

Books & long-form

There is no single canonical book on KEDA — the topic moves fast and lives in docs and blog posts. The following give the deepest context:

- "Production Kubernetes" by Josh Rosso et al. (O'Reilly) — chapter on autoscaling covers KEDA in the broader autoscaling landscape.
- "Cloud Native Patterns" by Cornelia Davis (Manning) — event-driven patterns that KEDA was built to serve.
- CNCF blog tag for KEDA: cncf.io/?s=keda

Practice / labs

- Spin up a local kind cluster, install KEDA, and run through the keda-samples repo end-to-end. About a half-day investment, pays off forever.
 - Recreate Example 1 (RabbitMQ) and intentionally break things: stop RabbitMQ, expire credentials, set absurd thresholds. Observe how KEDA behaves.
 - Write a tiny external scaler in any language. The gRPC contract is small and walking through it teaches you exactly how the system works internally.
- Writing external scalers: keda.sh/docs/2.19/concepts/external-scalers

Final tip

Start small. Pick one event-driven workload in a non-critical environment, switch it from a CPU HPA to a KEDA ScaledObject on the real signal, watch it for a week. The cost savings and the better behavior will sell KEDA inside your organization more effectively than any document — including this one — ever could.

About This Guide

This document is part of the [d-someone / tech-guide](#) project — a curated, open collection of practical technical guides authored and maintained on GitHub.

Feedback keeps these guides accurate and useful. If you spot something wrong, want a new topic covered, or have a question, please open an issue using one of the pre-filled templates below — it takes less than a minute.



Found a mistake?

Submit a correction with the dedicated template: [open a correction issue →](#)



Want a guide on a different topic?

Request a new guide here: [open a guide request →](#)



Have a question?

Ask in the question tracker: [open a question issue →](#)

— d-someone · github.com/d-someone/tech-guide